

1. Personal Identification

- **Full Name:** Jainil Sandip Patel
- **Date of Birth:** 25-02-2006
- **Location:** Ahmedabad, India

2. Education

- **Degree:** B.Tech in Computer Science and Engineering (Major)
- **Minor:** Adaptive AI
- **University:** Nirma University
- **Semester:** 6th
- **Minor Coursework Areas:**
 - Computer Vision using Deep Learning
 - Natural Language Processing

3. Contact Information

- **Secondary Email:** jainilpatel2222@gmail.com
- **Contact no.:** 7567141068

4. Skills

- **Programing Languages:** Python, Java, JavaScript
- **Frameworks & Tools:** PyTorch, LangChain, LangGraph, React, Fast API
- **Concepts :** Deep Learning , Machine Learning , Gen AI , OOP , DSA , DBMS , SQL
- **Cloud :** AWS Certified Solutions architect Associate

6. Profiles

- Github: <https://github.com/JainilPatel2502>
- LinkedIn: <https://www.linkedin.com/in/jainil-patel-952564278/>
- Leetcode: https://leetcode.com/u/Jainil_Patel2502/
- HuggingFace: <https://huggingface.co/Jainil2502>

7. Areas of Interest

- AI Model Training
- GenAI Systems
- Agentic AI
- API-based AI integrations (MCP, tool calling, etc.)
- AWS Solutions Architect
- GCP

8. Future Goals (Till End of 2026)

- **Distributed training** and build large-scale deep learning systems across multi-GPU and multi-node setups.
- **Reinforcement learning**, applying RL to intelligent decision-making and autonomous systems.
- **Develop deep MLOps expertise**, including model deployment, monitoring, CI/CD, and full ML lifecycle management.

9. Cloud Skills in AWS

- Jainil Patel earned the **AWS Certified Solutions Architect – Associate** certification with a score of **871/1000 on his first attempt**. His preparation spanned **two months** and was primarily based on the **Adrian Cantrill AWS Solutions Architect Associate course**, which emphasizes deep conceptual understanding of cloud architecture. This

certification reflects his ability to understand cloud design principles and apply them effectively to real-world infrastructure and architecture scenarios.

- Even this portfolio website is hosted on aws.

Project 1 — AutoCAD MCP Server (Work in Progress)

Project Progress Report

AutoCAD Agent — AI-Powered Floor Plan Generation (LangGraph-Based)

Aim

To develop an AI-powered AutoCAD automation agent that converts **natural language architectural prompts** into **complete AutoCAD floor plan drawings**. The system generates AutoCAD-ready geometry by planning architectural layouts, generating Python CAD code, executing it in AutoCAD, and summarizing outputs for better interaction.

System Overview

AutoCAD Agent is a LangGraph-based agent that transforms floor plan descriptions written in plain English into AutoCAD drawings.

It demonstrates how LLMs can orchestrate CAD primitives to generate architectural plans automatically.

Important Disclaimer

This is **NOT a production-grade agent** and does not claim complete automation of AutoCAD drafting.

It is a **prototype/research step** towards AI-assisted CAD workflows.

Agent Pipeline (LangGraph Workflow)

User Prompt → Planner → Coder → Executor → Summarizer

Key Features Implemented

1) Natural Language → AutoCAD Drawing

Users provide prompts like:

"Create a living room with a door on the south wall and two windows facing east"
and the system generates the corresponding AutoCAD floor plan output.

2) Modular CAD Primitive System

Supports reusable architectural primitives such as:

- Walls
- Doors
- Windows
- Stairs
- Dimensions
- Room Tags

3) Door & Window Insertion with Wall Cutting

Doors and windows are inserted with correct CAD behavior:

- automatic wall cutting
- block-based insertion
- support for different window types (fixed / sliding)

4) Junction Cleaning (Geometry Cleanup)

Implements cleanup utilities for:

- **L-Junctions**
- **T-Junctions**

This improves CAD correctness and architectural connectivity.

5) Live AutoCAD Execution (COM Automation)

The generated CAD code executes directly in a running AutoCAD instance using COM-based automation.

CAD Modules & Utilities

CAD Primitives Modules

- `wall.py` — draw walls as closed polylines

- `door.py` — cut walls and insert door blocks
- `window.py` — cut walls and insert window blocks
- `stairs.py` — draw staircases with treads
- `dimension.py` — add aligned dimensions
- `room_tag.py` — label rooms with name & area

Utility Modules

- `vertex_cleaner.py` — clean L-junctions and T-junctions
 - `refresher.py` — force AutoCAD redraws
 - `block_manager.py` — create/manage door & window blocks
-

Current Status & Outcomes

- LangGraph workflow is functioning end-to-end
 - Successfully generates architectural floor plans inside AutoCAD from natural language prompts
 - Core architectural drawing elements are working:
 - walls
 - doors/windows insertion with wall cutting
 - junction cleanup
 - dimensioning and room tagging
 - The project is currently active and continuously improving
-

Tech Stack

- **Python** — agent backend and CAD logic
 - **LangGraph** — workflow orchestration
 - **LangChain + Google GenAI** — LLM planning and code generation
 - **pyautocad** — AutoCAD COM-based automation
-

Repository Reference

GitHub repository:

<https://github.com/JainilPatel2502/AutoCAD-Agent-CodeGeneration>

Conclusion

AutoCAD MCP Server is a modular automation backend that bridges AutoCAD and modern AI/agent pipelines using FastMCP and Python-driven COM automation. The project provides a strong foundation for scalable AutoCAD tool invocation and is being actively enhanced with broader CAD command support and improved automation reliability.

Project 2 — MiniReasoner-70M (2025)

Overview

- **MiniReasoner-70M** is a **70M-parameter modern transformer model** trained from scratch on a local **RTX 3060 GPU** with 6GB VRAM.
 - The project demonstrates Jainil's hands-on experience with **full LLM training**: tokenizer → architecture → training → inference.
 - Purpose is **educational**, focusing on understanding the entire transformer pipeline—not competing with frontier models like GPT, Gemini, or DeepSeek.
 - Github: <https://github.com/JainilPatel2502/minireasoner-70m>
 - HuggingFace: <https://huggingface.co/Jainil2502/minireasoner-70m>
-

Objectives

- Build an end-to-end transformer implementation using **PyTorch**.
 - Train a custom model on **1.4B tokens** (OpenCodeReasoning synthetic dataset).
 - Experiment with modern transformer improvements and training optimizations.
 - Gain practical experience with constraints on a mid-range GPU (RTX 3060).
-

Model Architecture & Features

- **RMSNorm** instead of LayerNorm → improved training stability.
- **PreNorm architecture** → better gradient flow in deeper networks.
- **SwiGLU activation** (PaLM, LLaMA) → smoother, more expressive representations.
- **Bias-free & dropout-free architecture** → smaller, cleaner parameterization.

- **Attention head ratio = 2.66 rule** → balanced head dimensionality.
 - **FlashAttention** → memory-efficient, faster attention mechanism.
 - **RoPE (Rotary Positional Embeddings)** → improved positional generalization.
 - **Tied input–output embeddings** → parameter efficiency.
 - **Custom tokenizer** trained from scratch (vocab size **8192**).
 - **Mixed precision (bfloat16) training + gradient clipping**.
 - **Cosine LR schedule with warmup (1000 steps)**.
 - **Checkpointing every 500 steps** for safe recovery.
 - **TensorBoard** visualization for loss/perplexity.
 - **Streaming DataLoader** for efficient memory usage.
-

Training Details

- **Dataset:** 1.4B tokens from [nvidia/OpenCodeReasoning](#).
 - **Hardware:** Single **RTX 3060 GPU** (12GB VRAM).
 - **Precision:** bfloat16 + gradient scaling.
 - **Tokenizer:** Custom BPE tokenizer built from scratch.
 - **Challenges:** memory constraints, longer training cycles, manual optimization of batch size & sequence length.
-

Results

- The model is **not competitive**, and **does not generate good reasoning outputs** (expected).
- The success of the project lies in **end-to-end understanding**, not SOTA performance.
- Demonstrated ability to train a working transformer pipeline completely from scratch.

Acknowledgements

- Inspired by **Andrej Karpathy's Zero to Hero** series.

Project 3 — Neuro Builder (2025)

Overview

Neuro Builder is an **AI-powered GUI tool** that lets users design, train, and monitor neural networks **without writing any code**.

Users upload their dataset, configure the neural network through an interactive playground, and train the model with real-time logs and performance tracking.

Objectives

- Build an neural network training tool accessible to non-coders.
 - Provide an interactive playground for defining model architecture.
 - Enable real-time monitoring of training metadata and model performance.
 - Integrate a PyTorch backend with a modern React UI.
-

Key Features

Dataset Loading

- Upload preprocessed datasets (CSV).
- Supports **classification** and **regression** tasks.

Neural Network Playground

- Choose number of layers and neurons per layer.
- Select activation functions (ReLU, Sigmoid, Tanh, etc.).
- Configure loss function, optimizer, learning rate, epochs, and batch size.

- Fully flexible model architecture design.

Live Training Logs

- Real-time monitoring using WebSockets.
- Track loss curves and performance metrics while training.

Customization & Flexibility

- Build architectures interactively.
 - Adjust hyperparameters on the fly.
 - Experiment with different model configurations easily.
-

How It Works

1. Upload dataset (Preprocessed)
 2. Select problem type (classification or regression).
 3. Build model using the playground.
 4. Train with real-time log streaming.
 5. Evaluate metrics (accuracy, MSE, etc.).
-

Tech Stack

- **Frontend:** React.js
- **Backend:** PyTorch

- **Real-time Logging:** WebSockets

Demo Video

<https://github.com/user-attachments/assets/ea6bf56a-2142-4dd9-8eda-9da3c8569b37>

Backend Repository

<https://github.com/JainilPatel2502/NeuroBuilder-Backend>

Frontend Repository

<https://github.com/JainilPatel2502/NeuroBuilder-Frontend>

Project 4 — EDA Tool Frontend (2024)

Overview

An **Exploratory Data Analysis (EDA) web application** designed for ML practitioners and data scientists to quickly explore datasets through **univariate, bivariate, and multivariate visualizations**.

Users simply upload a dataset and instantly generate a wide variety of plots — no coding or setup required.

- **Live Demo:** <https://data-insider.onrender.com>
 - **Repo:** https://github.com/JainilPatel2502/EDA_Tool_Frontend
-

Key Features

- **Interactive EDA Dashboard**
 - Univariate, bivariate, and multivariate plot generation
 - Instant visualizations from uploaded CSV datasets
- **Wide Range of Visualizations**
 - Bar, Line, Pie
 - Sankey Diagram
 - Contour & Density Plots
 - 3D Scatter
 - Dot & Bubble
 - Mosaic & Pair Plots
 - Radar Charts
 - Pareto Plots

- **No installation required** — fully browser-based
 - **Clean, intuitive UI** (designed with guidance from Claude)
-

Tech Stack

- **Frontend:** React + Plotly.js
- **Backend:** FastAPI